LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4

VIEWMODEL AND DEBUGGING

Oleh:

Hafiz Perdana NIM. 2410817210027

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Hafiz Perdana
NIM : 2410817210027

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 6
A. Source Code 7
B. Output Program 22
C. Pembahasan 23
SOAL 2 29
A. Pembahasan 29
TAUTAN GIT 30

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Test Debugging Compose..... 22
Gambar 2. Screenshot Hasil Test Debugging XML..... 23

DAFTAR TABEL

Tabel 1. Source Code MyApplication.kt Soal 1 7
Tabel 2. Source Code GameViewModel.kt Soal 1 7
Tabel 3. Source Code GameViewModelFactory.kt Soal 1 8
Tabel 4. Source Code HomeScreen.kt Soal 1 9
Tabel 5. Source Code DetailScreen.kt Soal 1 13
Tabel 6. Source Code MainApplication.kt Soal 1 15
Tabel 7. Source Code GameViewModel.kt Soal 1 15
Tabel 8. Source Code GameViewModelFactory.kt Soal 1 17
Tabel 9. Source Code HomeFragment.kt Soal 1 17
Tabel 10. Source Code DetailFragment.kt Soal 1 20

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
d. Install dan gunakan library Timber untuk logging event berikut:
e. Log saat data item masuk ke dalam list
f. Log saat tombol Detail dan tombol Explicit Intent ditekan
g. Log data dari list yang dipilih ketika berpindah ke halaman Detail e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

COMPOSE

MyApplication.kt

Tabel 1. Source Code MyApplication.kt Soal 1

```
package com.example.modul3compose

import android.app.Application
import android.content.pm.ApplicationInfo
import timber.log.Timber

class MyApplication : Application() {
    override fun onCreate() {
        super.onCreate()

        val isDebug = (applicationInfo.flags and ApplicationInfo.FLAG_DEBUGGABLE) != 0

        if (isDebug) {
            Timber.plant(Timber.DebugTree())
        }
    }
}
```

Package viewmodel - GameViewModel.kt

Tabel 2. Source Code GameViewModel.kt Soal 1

```
package com.example.modul3compose.viewmodel

import androidx.lifecycle.ViewModel
import com.example.modul3compose.data.GameData
import com.example.modul3compose.model.Game
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import timber.log.Timber

class GameViewModel(private val username: String) : ViewModel() {

    private val _games = MutableStateFlow<List<Game>>(emptyList())
    val games: StateFlow<List<Game>> = _games.asStateFlow()

```

```
    private val _selectedGame = MutableStateFlow<Game?>(null)
    val selectedGame: StateFlow<Game?> = _selectedGame.asStateFlow()

    init {
        Timber.d("User: $username - Loading games")
        loadGames()
    }

    private fun loadGames() {
        val data = GameData.gameList
        _games.value = data
        Timber.d("Game list loaded: ${data.size} items")
    }

    fun onDetailClicked(game: Game) {
        Timber.d("Tombol Detail ditekan untuk: ${game.title}")

        Timber.d("Navigasi ke halaman Detail dengan data: ID=${game.id}, Judul=${game.title}")

        _selectedGame.value = game
    }

    fun onExplicitIntentClicked(url: String) {
        Timber.d("Tombol Explicit Intent (Browser) ditekan untuk URL: $url")
    }

    fun clearSelection() {
        _selectedGame.value = null
    }
}
```

Package viewmodel - GameViewModelFactory.kt

Tabel 3. Source Code GameViewModelFactory.kt Soal 1

```
package com.example.modul3compose.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider

class GameViewModelFactory(private val username: String) : ViewModelProvider.Factory {
```

```
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(GameViewModel::class.java)) {
            @Suppress("UNCHECKED_CAST")
            return GameViewModel(username) as T
        }
        throw IllegalArgumentException("Unknown ViewModel class")
    }
}
```

Package screen - HomeScreen.kt

Tabel 4. Source Code HomeScreen.kt Soal 1

```
package com.example.modul3compose.ui.screen

import android.content.Intent
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.*
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.*
import androidx.compose.ui.res.*
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.*
import androidx.core.net.toUri
import com.example.modul3compose.R
import com.example.modul3compose.data.GameData
import com.example.modul3compose.model.Game

import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.modul3compose.viewmodel.GameViewModel
import com.example.modul3compose.viewmodel.GameViewModelFactory
```

```

@Composable
fun HomeScreen(
    navController: androidx.navigation.NavController,
    viewModel: GameViewModel = viewModel(factory = GameViewModelFactory("Hafiz Perdana"))
) {
    val games by viewModel.games.collectAsState()
    val selectedGame by viewModel.selectedGame.collectAsState()

    selectedGame?.let { game ->
        navController.navigate("detail/${game.id}")
        viewModel.clearSelection()
    }

    Scaffold(
        topBar = {
            Surface(color = Color.Black, shadowElevation = 4.dp) {
                Row(
                    modifier = Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal = 16.dp),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(stringResource(R.string.app_name), style = MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold, modifier = Modifier.weight(1f))
                    IconButton(onClick = { navController.navigate("settings") }) {
                        Icon(Icons.Default.Settings, null, tint = Color.White)
                    }
                }
            }
        }
    ) { p ->
        LazyColumn(modifier = Modifier.fillMaxSize().padding(p)) {
            item {

Text(stringResource(R.string.title_highlight), modifier = Modifier.padding(16.dp), fontWeight = FontWeight.SemiBold)
```

```
                LazyRow(contentPadding = PaddingValues(horizontal = 16.dp), horizontalArrangement = Arrangement.spacedBy(12.dp)) {
                    items(games) { game ->
                        Card(
                            onClick = { viewModel.onDetailClicked(game) },
                            modifier = Modifier.size(200.dp, 120.dp),
                            shape = RoundedCornerShape(12.dp)
                        ) {
                            Image(painterResource(game.imageRes), null, contentScale = ContentScale.Crop, modifier = Modifier.fillMaxSize())
                        }
                    }
                }
                Spacer(Modifier.height(24.dp))
            }
            items(games) { game ->
                GameListItem(
                    game = game,
                    onDetail = { viewModel.onDetailClicked(game) },
                    onBrowser = { url -> viewModel.onExplicitIntentClicked(url) }
                )
            }
        }
    }
}

@Composable
fun GameListItem(game: Game, onDetail: () -> Unit, onBrowser: (String) -> Unit) {
    val ctx = LocalContext.current
    val config = LocalConfiguration.current

    val lang = config.locales.get(0).language
    val isId = lang == "in" || lang == "id"

    Card(
        modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp).fillMaxWidth().height(175.dp),
        shape = RoundedCornerShape(16.dp),
```

```
        colors = CardDefaults.cardColors(containerColor = Color(0xFF1A1A1A))
    ) {
        Row(Modifier.padding(12.dp).fillMaxSize()) {
            Image(
                painterResource(game.imageRes), null,
                modifier = Modifier.size(100.dp, 150.dp).clip(RoundedCornerShape(12.dp)),
                contentScale = ContentScale.Crop
            )
            Column(Modifier.padding(start = 12.dp).fillMaxHeight()) {
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) {
                    Text(game.title, fontWeight = FontWeight.Bold, maxLines = 1, overflow = TextOverflow.Ellipsis, modifier = Modifier.weight(1f))
                    Text(game.year, style = MaterialTheme.typography.bodySmall, color = Color.Gray)
                }
                Text(
                    text = if (isId) game.plotId else game.plotEn,
                    style = MaterialTheme.typography.bodySmall, color = Color.LightGray,
                    maxLines = 2, overflow = TextOverflow.Ellipsis, modifier = Modifier.padding(top = 4.dp), lineHeight = 16.sp
                )
                Spacer(Modifier.weight(1f))
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.End) {
                    val blue = ButtonDefaults.buttonColors(containerColor = Color(0xFF2196F3))
                    Button(
                        onClick = {
                            onBrowser(game.url)
                            ctx.startActivity(Intent(Intent.ACTION_VIEW, game.url.toUri()))
                        },
                        colors = blue,
                        modifier = Modifier.height(36.dp)
                    ) {
```

```

Text(stringResource(R.string.btn_browser), fontSize = 11.sp)
                    }
                    Spacer(Modifier.width(8.dp))
                    Button(onClick = onDetail, colors = blue, modifier = Modifier.height(36.dp)) {
Text(stringResource(R.string.btn_detail), fontSize = 11.sp)
                    }
                }
            }
        }
    }
}
```

Package screen - DetailScreen.kt

Tabel 5. Source Code DetailScreen.kt Soal 1

```
package com.example.modul3compose.ui.screen

import androidx.compose.foundation.*
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.*
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalConfiguration
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.modul3compose.data.GameData

@Composable
fun DetailScreen(navController: androidx.navigation.NavController, gameId: Int?) {
    val game = GameData.gameList.find { it.id == gameId }
    val lang = LocalConfiguration.current.locales.get(0).language
```

```
    val isId = lang == "in" || lang == "id"

    Scaffold(
        topBar = {
            Surface(color = Color.Black, shadowElevation = 4.dp) {
Row(Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal = 8.dp), verticalAlignment = Alignment.CenterVertically) {
                    IconButton(onClick = { navController.popBackStack() }) {
Icon(Icons.AutoMirrored.Filled.ArrowBack, null, tint = Color.White)
                    }
                    Text(game?.title ?: "Detail", style = MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold, modifier = Modifier.padding(start = 8.dp))
                }
            }
        }
    ) { p ->
        game?.let {
Column(Modifier.fillMaxSize().padding(p).verticalScroll(rememberScrollState()).padding(16.dp)) {
                Image(
                    painterResource(it.imageRes), null,
                    modifier = Modifier.fillMaxWidth().height(260.dp).clip(RoundedCornerShape(16.dp)),
                    contentScale = ContentScale.Crop
                )


Row(Modifier.fillMaxWidth().padding(vertical = 20.dp), horizontalArrangement = Arrangement.SpaceBetween) {
                    Text(it.title, style = MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.Bold, modifier = Modifier.weight(1f))
                    Text(it.year, style = MaterialTheme.typography.titleLarge, color = Color.Gray)
                }

                Text("Plot", fontWeight = FontWeight.Bold)
```

```
                Text(if (isId) it.plotId else it.plotEn, modifier = Modifier.padding(top = 8.dp))
            }
        }
    }
}
```

XML

MainApplication.kt

Tabel 6. Source Code MainApplication.kt Soal 1

```
package com.example.modul3xml

import android.app.Application
import android.content.pm.ApplicationInfo
import timber.log.Timber

class MainApplication : Application() {
    override fun onCreate() {
        super.onCreate()

        val isDebug = (applicationInfo.flags and ApplicationInfo.FLAG_DEBUGGABLE) != 0

        if (isDebug) {
            Timber.plant(Timber.DebugTree())
        }
    }
}
```

Package viewmodel - GameViewModel.kt

Tabel 7. Source Code GameViewModel.kt Soal 1

```
package com.example.modul3xml.ui.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.modul3xml.data.Game
import com.example.modul3xml.data.GameData
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import timber.log.Timber

```

```
class GameViewModel(private val category: String) : ViewModel() {

    private val _gameList = MutableStateFlow<List<Game>>(emptyList())
    val gameList: StateFlow<List<Game>> = _gameList.asStateFlow()

    private val _navigateToDetail = MutableStateFlow<Game?>(null)
    val navigateToDetail: StateFlow<Game?> = _navigateToDetail.asStateFlow()

    init {
        loadGames()
    }

    private fun loadGames() {
        viewModelScope.launch {
            val data = GameData.gameList
            _gameList.value = data
            Timber.d("Data game berhasil dimuat ke ViewModel: ${data.size} item")
        }
    }

    fun onDetailClicked(game: Game) {
        Timber.d("Tombol Detail ditekan untuk game: ${game.title}")
        _navigateToDetail.value = game
    }

    fun onExplicitIntentClicked(game: Game) {
        Timber.d("Tombol Explicit Intent ditekan untuk game: ${game.title}")
    }

    fun onDetailNavigated() {
        _navigateToDetail.value = null
    }

    fun getCategory(): String = category
}
```

Tabel 8. Source Code GameViewModelFactory.kt Soal 1

```
1  package com.example.modul3xml.ui.viewmodel
2
3  import androidx.lifecycle.ViewModel
4  import androidx.lifecycle.ViewModelProvider
5
6  class GameViewModelFactory(private val category: String) :
   ViewModelProvider.Factory {
7      @Suppress("UNCHECKED_CAST")
8      override fun <T : ViewModel> create(modelClass:
   Class<T>): T {
9          if
   (modelClass.isAssignableFrom(GameViewModel::class.java)) {
10             return GameViewModel(category) as T
11         }
12         throw IllegalArgumentException("Unknown ViewModel
   class")
13     }
14 }
```

Package home - HomeFragment.kt

Tabel 9. Source Code HomeFragment.kt Soal 1

```
1  package com.example.modul3xml.ui.home
2
3  import android.content.Intent
4  import android.net.Uri
5  import android.os.Bundle
6  import android.view.LayoutInflater
7  import android.view.View
8  import android.view.ViewGroup
9  import androidx.fragment.app.Fragment
10 import androidx.fragment.app.viewModels
11 import androidx.lifecycle.Lifecycle
12 import androidx.lifecycle.lifecycleScope
13 import androidx.lifecycle.repeatOnLifecycle
14 import androidx.navigation.fragment.findNavController
15 import androidx.recyclerview.widget.LinearLayoutManager
16 import com.example.modul3xml.R
17 import
   com.example.modul3xml.databinding.FragmentHomeBinding
18 import com.example.modul3xml.ui.viewmodel.GameViewModel
19 import
   com.example.modul3xml.ui.viewmodel.GameViewModelFactory
20 import kotlinx.coroutines.launch
21 import timber.log.Timber
```

```

class HomeFragment : Fragment() {

    private var _binding: FragmentHomeBinding? = null

    private val viewModel: GameViewModel by viewModels {
        GameViewModelFactory("All Games")
    }

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        _binding = FragmentHomeBinding.inflate(inflater, container, false)
        return _binding?.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        _binding?.let { binding ->
            setupMenu(binding)
            setupRecyclerViews(binding)
            observeViewModel(binding)
        }
    }

    private fun setupMenu(binding: FragmentHomeBinding) {
        binding.toolbarHome.setOnMenuItemClickListener { item ->
            when (item.itemId) {
                R.id.menu_settings -> {
                    findNavController().navigate(R.id.action_homeFragment_to_settingsFragment)
                    true
                }
                else -> false
            }
        }
        binding.toolbarHome.title = getString(R.string.app_name)
    }

```

```
    private fun setupRecyclerViews(binding: FragmentHomeBinding) {
        binding.rvHighlight.layoutManager = LinearLayoutManager(requireContext(), LinearLayoutManager.HORIZONTAL, false)
        binding.rvGames.layoutManager = LinearLayoutManager(requireContext())
    }

    private fun observeViewModel(binding: FragmentHomeBinding) {
        viewLifecycleOwner.lifecycleScope.launch {
viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
                launch {
                    viewModel.gameList.collect { games ->
                        if (games.isNotEmpty()) {
                            binding.rvHighlight.adapter = HighlightAdapter(games) { game ->
viewModel.onDetailClicked(game)
                            }
                            binding.rvGames.adapter = GameListAdapter(
                                games = games,
                                onBrowserClick = { game ->
viewModel.onExplicitIntentClicked(game)
startActivity(Intent(Intent.ACTION_VIEW, Uri.parse(game.url)))
                                },
                                onDetailClick = { game ->
viewModel.onDetailClicked(game)
                                }
                            )
                        }
                    }
                }

                launch {
                    viewModel.navigateToDetail.collect { game ->
                        game?.let {
                            val bundle = Bundle().apply {
```

```
                                putInt("gameId", it.id)
                            }

findNavController().navigate(R.id.action_homeFragment_to_detailFragment, bundle)
                            viewModel.onDetailNavigated()
                        }
                    }
                }
            }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }

    override fun onResume() {
        super.onResume()
        _binding?.let { binding ->
            binding.toolbarHome.title = getString(R.string.app_name)
        }
    }
}
```

Package detail - DetailFragment.kt

Tabel 10. Source Code DetailFragment.kt Soal 1

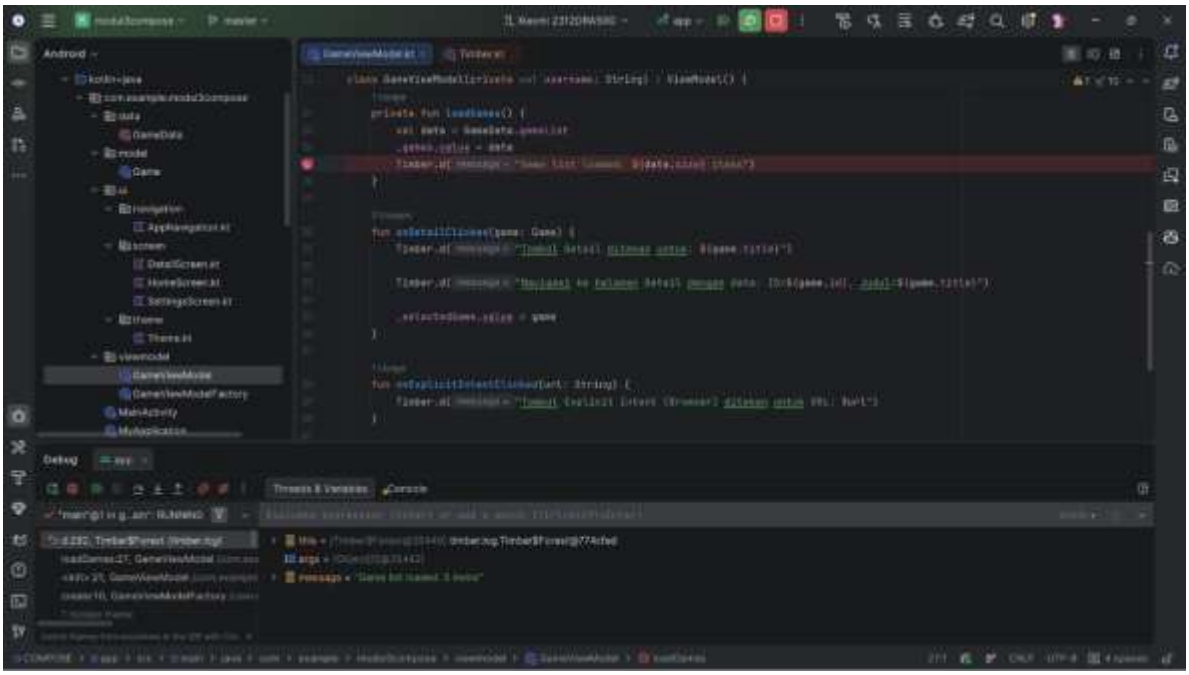
```
package com.example.modul3xml.ui.detail

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import com.example.modul3xml.R
import com.example.modul3xml.data.GameData
import com.example.modul3xml.databinding.FragmentDetailBinding
import com.example.modul3xml.util.LanguageManager
import timber.log.Timber

class DetailFragment : Fragment() {
```

```

    private var _binding: FragmentDetailBinding? = null


    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        _binding = FragmentDetailBinding.inflate(inflater, container, false)
        return _binding?.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        _binding?.let { binding ->
            setupToolbar(binding)
            bindGameData(binding)
        }
    }

    private fun setupToolbar(binding: FragmentDetailBinding) {
        binding.toolbarDetail.setNavigationOnClickListener {
            val gameId = arguments?.getInt("gameId") ?: -1
            Timber.d("Kembali dari halaman Detail untuk game ID: $gameId")
            findNavController().popBackStack()
        }
    }

    private fun bindGameData(binding: FragmentDetailBinding) {
        val gameId = arguments?.getInt("gameId") ?: -1
        val game = GameData.findGameById(gameId)

        if (game == null) {
            Timber.w("Game dengan id=$gameId tidak ditemukan!")
            binding.toolbarDetail.title = getString(R.string.title_detail)
            return
        }
```

```

        Timber.d("Menampilkan detail game: ${game.title} (ID: ${game.id})")

        val isIndonesian = LanguageManager.isIndonesian(requireContext())

        binding.toolbarDetail.title = game.title
        binding.imageDetail.setImageResource(game.imageRes)
        binding.textDetailTitle.text = game.title
        binding.textDetailYear.text = game.year
        binding.textDetailPlot.text = if (isIndonesian) game.plotId else game.plotEn
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

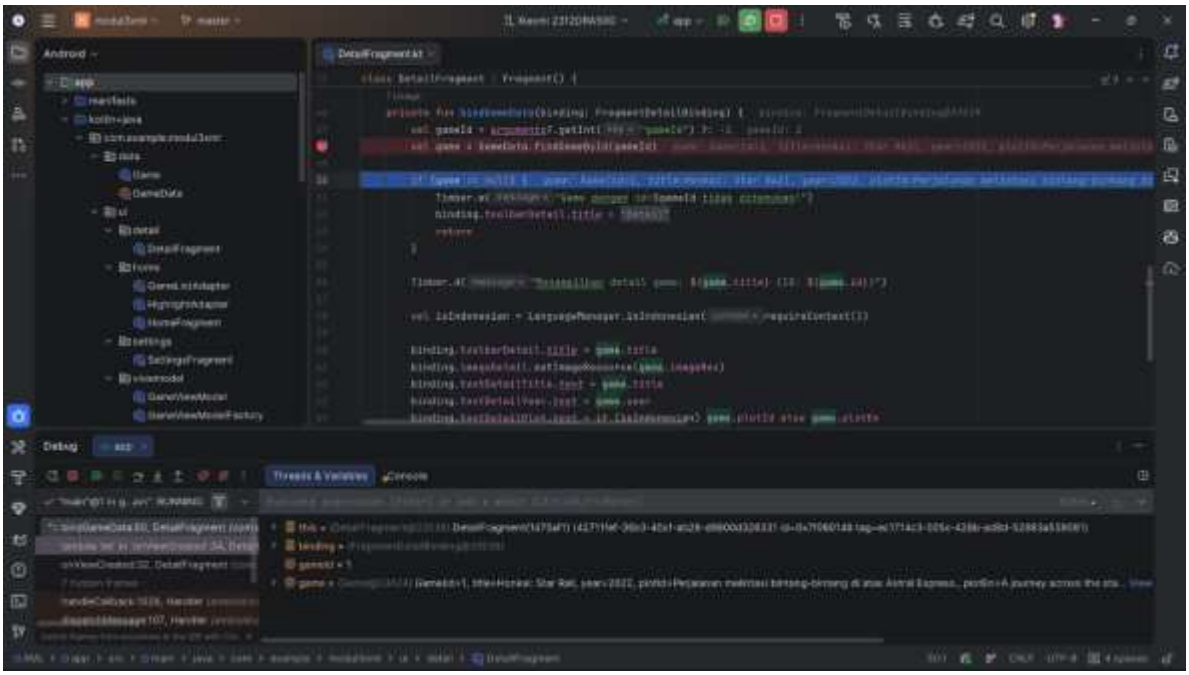
B. Output Program

COMPOSE



Gambar 1. Screenshot Hasil Test Debugging Compose

XML



Gambar 2. Screenshot Hasil Test Debugging XML

C. Pembahasan

COMPOSE

MyApplication.kt

Kelas ‘MyApplication’ diimplementasikan sebagai turunan dari kelas ‘Application’ untuk mengelola konfigurasi pada tingkat global. Metode ‘onCreate()’ di-override agar library Timber dapat diinisialisasi tepat saat proses aplikasi pertama kali dijalankan. Pendeteksian status mode pengembangan dilakukan melalui operasi bitwise pada ‘applicationInfo.flags’ dengan konstanta ‘ApplicationInfo.FLAG_DEBUGGABLE’.

Jika aplikasi terdeteksi berjalan dalam mode debug, maka ‘Timber.plant(Timber.DebugTree())’ dieksekusi untuk mengaktifkan pohon debug. Penempatan inisialisasi pada kelas ini mengakibatkan seluruh aktivitas pencatatan log dapat dipantau melalui jendela Logcat di Android Studio selama tahap pengerjaan, tanpa membebani performa atau memunculkan log pada versi rilis aplikasi.

Package viewmodel - GameViewModel.kt

Pengelolaan data dan logika bisnis dipisahkan dari lapisan antarmuka melalui kelas 'GameViewModel'. Variabel 'MutableStateFlow' diterapkan pada properti '_games' dan '_selectedGame' untuk menyimpan data list game serta status navigasi secara reaktif. Penggunaan 'asStateFlow()' dilakukan untuk mengekspos data tersebut sebagai 'StateFlow' yang bersifat read-only agar integritas data tetap terjaga dari intervensi langsung di sisi UI.

Pencatatan log melalui Timber disisipkan pada beberapa titik krusial untuk melacak aktivitas aplikasi. Pada blok 'init', fungsi 'loadGames()' dipanggil dan pesan log dicatat saat data berhasil masuk ke dalam list. Log juga ditambahkan pada fungsi 'onDetailClicked()' untuk merekam detail objek game yang dipilih (ID dan judul) sebelum proses navigasi dipicu. Selain itu, fungsi 'onExplicitIntentClicked()' juga memuat log saat tombol browser ditekan untuk mencatat URL yang diakses. Seluruh status data di dalam ViewModel ini tetap bertahan meskipun terjadi perubahan konfigurasi pada perangkat.

Package viewmodel - GameViewModelFactory.kt

Kelas 'GameViewModelFactory' dibuat dengan mengimplementasikan antarmuka 'ViewModelProvider.Factory'. Penggunaan factory ini diterapkan karena kelas 'GameViewModel' memerlukan parameter 'username' bertipe 'String' pada konstruktornya. Pembuatan instansi ViewModel secara standar tidak mendukung pengiriman argumen kustom, sehingga kelas ini bertindak sebagai jembatan untuk menyuntikkan data tersebut.

Metode 'create()' di-override untuk memvalidasi apakah kelas ViewModel yang diminta adalah 'GameViewModel'. Jika validasi berhasil, instansi dibuat dengan menyertakan parameter nama pengguna ("Hafiz Perdana").

Package screen - HomeScreen.kt

Integrasi antara lapisan antarmuka dan sumber data pada file 'HomeScreen.kt' dilakukan melalui observasi terhadap 'StateFlow' dari ViewModel. Fungsi 'collectAsState()' digunakan untuk mengubah aliran data menjadi state yang dapat dibaca oleh Compose, sehingga layar akan melakukan pembaruan tampilan (recomposition) secara otomatis setiap kali terjadi perubahan data list atau status klik.

Interaksi pada item list diarahkan ke fungsi ‘onDetailClicked()’ di ViewModel. Untuk menangani perpindahan halaman, digunakan blok ‘selectedGame?.let { ... }’ sebagai mekanisme keamanan agar perintah navigasi hanya dijalankan jika data item yang dipilih tidak bernilai null. Tampilan list disusun menggunakan komponen ‘LazyColumn’ yang datanya bersumber langsung dari ViewModel.

Package screen - DetailScreen.kt

Halaman ‘DetailScreen.kt’ difokuskan pada presentasi rincian informasi game berdasarkan parameter ‘gameId’ yang diterima dari komponen navigasi. Data objek game dicari di dalam list melalui fungsi ‘find { it.id == gameId }’ untuk mencocokkan identitas unik item. Informasi yang berhasil ditemukan kemudian dipetakan ke dalam komponen UI seperti ‘Image’ untuk gambar dan ‘Text’ untuk judul serta deskripsi.

Penentuan bahasa tampilan pada bagian plot dilakukan dengan mengakses ‘LocalConfiguration’ untuk mendeteksi bahasa sistem yang sedang digunakan (Indonesia atau Inggris).

Debugger, Step Into, Step Over, dan Step Out

Penggunaan fitur Debugger di Android Studio dilakukan untuk melakukan inspeksi variabel secara real-time saat aplikasi sedang dieksekusi. Sebuah breakpoint diletakkan pada baris pencatatan log di dalam fungsi ‘loadGames()’ pada kelas ‘GameViewModel’

Step Into digunakan untuk masuk ke dalam detail eksekusi kode internal pada file ‘Timber.kt’. Berdasarkan hasil pengujian, ditemukan bahwa variabel ‘message’ terbaca berisi teks "Game list loaded: 5 items", sesuai dengan jumlah data yang ada di dalam list.

Step Over digunakan untuk mengeksekusi baris kode berikutnya di file yang sama tanpa masuk ke dalam fungsi lain yang dipanggil, sehingga alur baris demi baris dapat dipantau dengan jelas.

Step Out digunakan untuk keluar dari fungsi internal library dan segera kembali ke alur utama kode pada ‘GameViewModel’.

XML

MainApplication.kt

Kelas ‘MainApplication’ diimplementasikan sebagai turunan dari kelas ‘Application’ untuk menangani konfigurasi pada level global saat aplikasi pertama kali dijalankan. Metode ‘onCreate()’ di-override agar library Timber dapat diinisialisasi secara otomatis. Pendeteksian status mode pengembangan dilakukan melalui operasi bitwise pada ‘applicationInfo.flags’ dengan konstanta ‘ApplicationInfo.FLAG_DEBUGGABLE’.

Jika aplikasi terdeteksi berjalan dalam mode debug, maka perintah ‘Timber.plant(Timber.DebugTree())’ dieksekusi untuk mengaktifkan pohon debug pada Logcat.

Package viewmodel - GameViewModel.kt

Pemisahan logika manajemen data dari lapisan antarmuka dilakukan melalui kelas ‘GameViewModel’. Variabel ‘MutableStateFlow’ diterapkan pada properti ‘_gameList’ dan ‘_navigateToDetail’ untuk menyimpan status data list serta status navigasi secara reaktif. Penggunaan ‘asStateFlow()’ dilakukan untuk mengekspos data tersebut sebagai ‘StateFlow’ yang bersifat read-only agar integritas data tetap terjaga dari intervensi langsung di sisi UI.

Pencatatan log melalui Timber disisipkan pada blok ‘init’ saat fungsi ‘loadGames()’ dipanggil untuk mencatat jumlah item yang masuk ke dalam list. Log juga ditambahkan pada fungsi ‘onDetailClicked()’ untuk merekam detail game yang dipilih serta pada ‘onExplicitIntentClicked()’ untuk mencatat pemicuan tautan eksternal ke browser.

Package viewmodel - GameViewModelFactory.kt

Kelas ‘GameViewModelFactory’ dibuat dengan mengimplementasikan antarmuka ‘ViewModelProvider.Factory’ untuk menangani pembuatan instansi ViewModel yang memiliki parameter kustom. Penggunaan factory ini diperlukan karena secara standar, sistem tidak mendukung pengiriman argumen ‘String’ langsung ke dalam konstruktor ‘GameViewModel’.

Metode ‘create()’ di-override untuk memvalidasi kesesuaian kelas ViewModel yang diminta. Jika valid, instansi dibuat dengan menyertakan parameter kategori (misalnya “All Games”).

Package home - HomeFragment.kt

Integrasi antara antarmuka pengguna dan sumber data pada ‘HomeFragment’ dilakukan melalui observasi terhadap ‘StateFlow’ dari ViewModel menggunakan ‘lifecycleScope’ dan ‘repeatOnLifecycle’. Data list game yang diterima kemudian dialirkan ke dalam adapter RecyclerView untuk ditampilkan pada layar. Penanganan klik item diarahkan ke fungsi di ViewModel untuk memperbarui status navigasi, yang kemudian direspons oleh Fragment untuk memicu perpindahan halaman.

Seluruh akses terhadap objek binding dilakukan menggunakan safe call dan blok ‘let’ untuk memastikan tidak terjadi kebocoran memori atau kegagalan aplikasi akibat akses pada objek yang bernilai null setelah view dihancurkan.

Package detail - DetailFragment.kt

Halaman ‘DetailFragment’ difokuskan pada presentasi rincian informasi game berdasarkan parameter ‘gameId’ yang diterima melalui arguments bundle. Identitas tersebut digunakan untuk mencari objek data yang sesuai di dalam sumber data list. Informasi yang ditemukan kemudian dipetakan ke dalam komponen UI seperti gambar, judul, dan deskripsi plot berdasarkan preferensi bahasa sistem.

Pencatatan log Timber diletakkan pada fungsi ‘bindGameData()’ untuk merekam informasi spesifik mengenai game yang sedang ditampilkan. Pemisahan tanggung jawab ini memastikan bahwa halaman detail hanya berfungsi sebagai penyaji data yang statusnya tetap sinkron dengan data yang telah dikelola pada level ViewModel.

Debugger, Step Into, Step Over, dan Step Out

Penggunaan fitur Debugger di Android Studio dilakukan untuk memantau nilai variabel secara langsung pada saat aplikasi dieksekusi. Sebuah breakpoint diletakkan pada baris kode pencarian data di dalam fungsi ‘bindGameData()’ untuk menghentikan alur program sejenak.

Step Into digunakan untuk melompat masuk ke dalam fungsi ‘findGameById()’ guna memeriksa proses internal pencarian item di dalam list data.

Step Over digunakan untuk mengeksekusi baris kode saat ini dan berpindah ke baris berikutnya di file yang sama untuk memantau nilai variabel ‘game’ setelah data berhasil diambil.

Step Out digunakan untuk keluar dari fungsi yang sedang diperiksa dan segera kembali ke baris pemanggil di kode utama.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Application class adalah kelas dasar dalam Android yang mewakili keseluruhan aplikasi itu sendiri. Kelas ini diinisialisasi pertama kali sebelum komponen lain seperti Activity atau Service dijalankan, sehingga cocok digunakan untuk melakukan inisialisasi global yang dibutuhkan sepanjang siklus hidup aplikasi.

Secara default, setiap aplikasi Android sebenarnya sudah menggunakan Application class bawaan dari sistem. Namun, developer dapat membuat kelas turunan (subclass) dari Application untuk menambahkan logika inisialisasi sendiri. Caranya cukup dengan membuat kelas baru yang meng-extend Application, lalu mendaftarkannya di AndroidManifest.xml melalui atribut android:name pada tag application.

Fungsi utama Application class adalah sebagai tempat menyimpan state atau data yang bersifat global dan perlu diakses oleh seluruh bagian aplikasi. Misalnya, inisialisasi library pihak ketiga seperti Timber umumnya dilakukan di sini agar tidak perlu diinisialisasi berulang kali di setiap Activity.

TAUTAN GIT

Berikut adalah tautan untuk repositori GitHub yang telah dibuat

https://github.com/Fiezz65/MODUL-MOBILE/tree/master/MODUL_4